SE4050 – Deep Learning

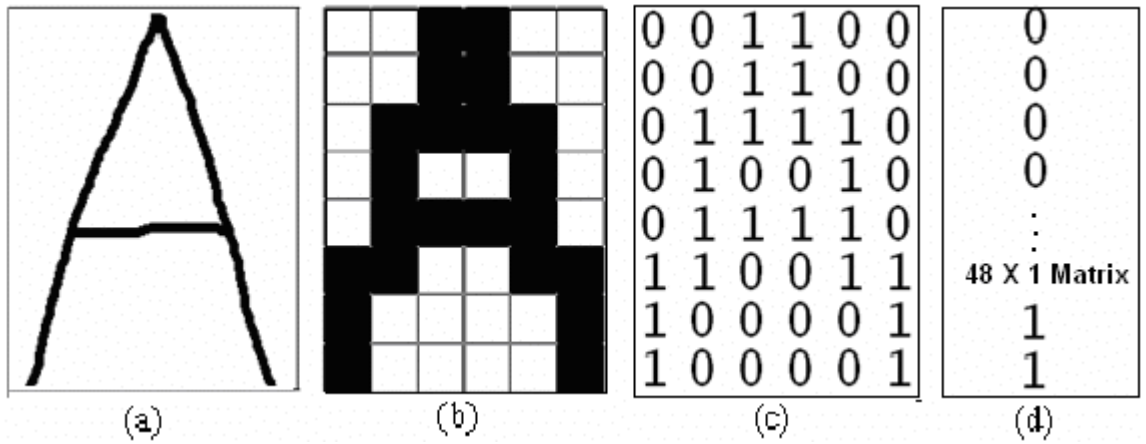
Week 4 – Convolutional Neural Networks

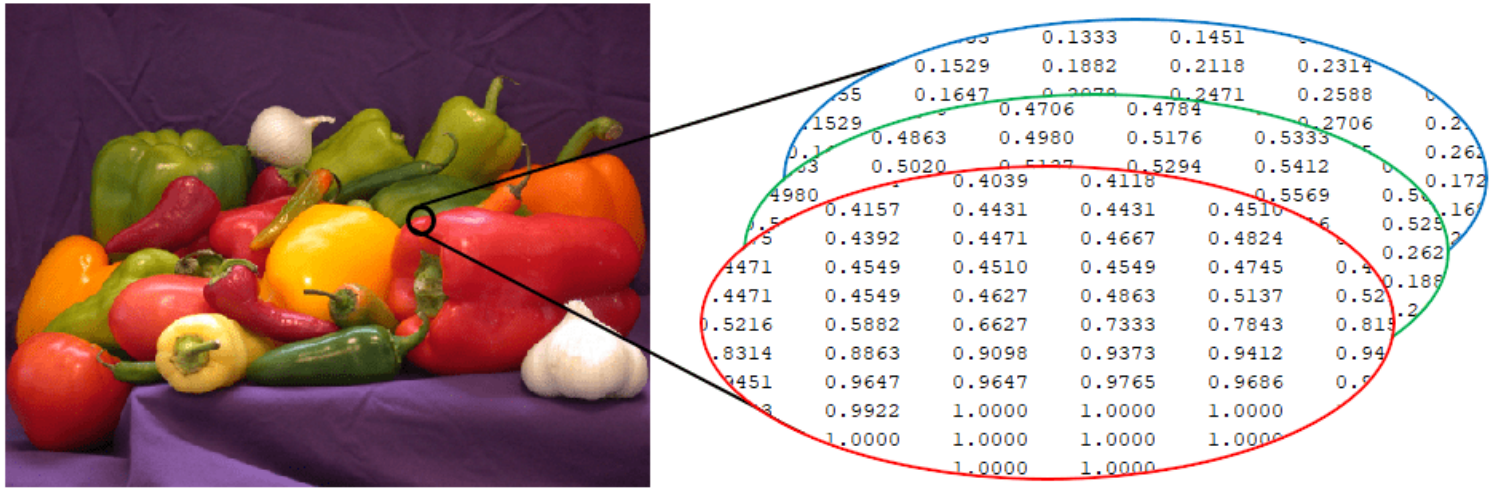
Dr. Mahima Weerasinghe, *MIET*

Content

- How computers perceive images
- Matrices and Neural Networks
- Convolution vs Multiplication
- Traditional Image Processing
- Working with CNNs

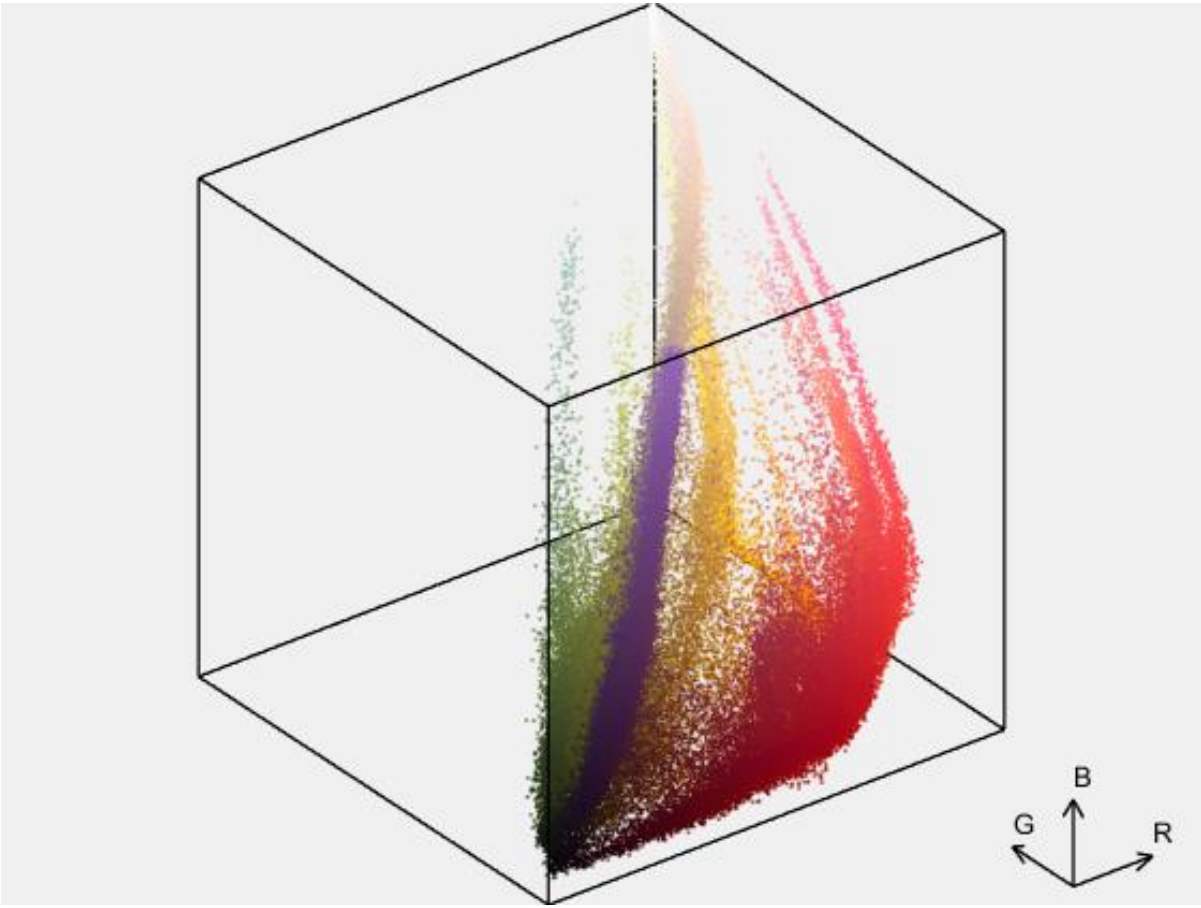
What is an Image in Computational Terms



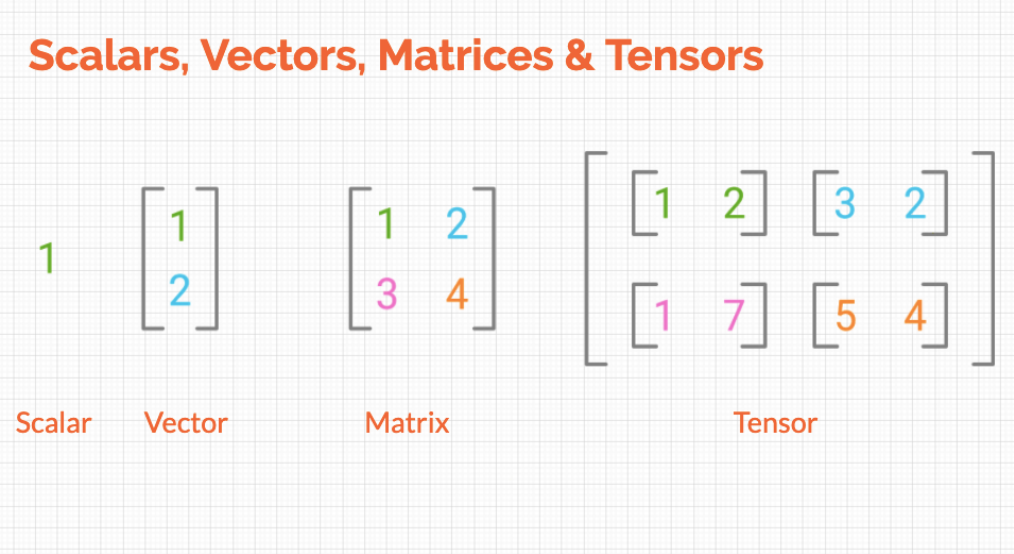


```
rgbImage = double(imread('peppers.png'));
[r, g, b] = imsplit(rgbImage);
xyz = [r(:), g(:), b(:)];
ptCloud = pointCloud(xyz);
pcshow(ptCloud)
```

b	384x512 double	384x512
g	384x512 double	384x512
r	384x512 double	384x512
rgbImage	384x512x3 double	384x512x3
xyz	196608x3 double	196608x3



Scalars, Vectors, Matrices



- Matrix addition

$$\begin{bmatrix} 2 & 1 \\ 7 & 9 \end{bmatrix} + \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 5 & 2 \\ 7 & 11 \end{bmatrix}$$

$$A_{2x2} + B_{2x2} = C_{2x2}$$

- Matrix subtraction

$$\begin{bmatrix} 2 & 1 \\ 7 & 9 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 5 & 6 \end{bmatrix}$$

- Matrix multiplication

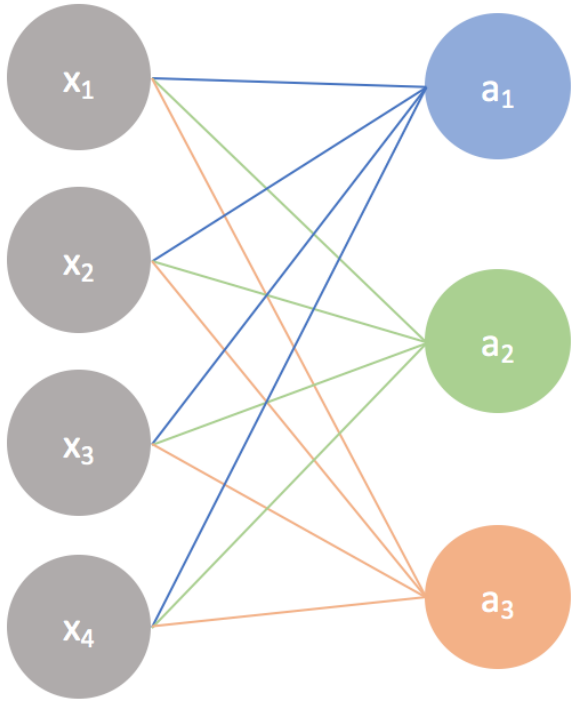
$$\begin{bmatrix} 2 & 1 \\ 7 & 9 \end{bmatrix} \mathrm{x} \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 3 \\ 26 & 27 \end{bmatrix}$$

$$A_{2x2} \mathrm{x} B_{2x2} = C_{2x2}$$

- Scalar multiplication

$$\frac{1}{8} \begin{bmatrix} 2 & 4 \\ 6 & 1 \end{bmatrix} = \begin{bmatrix} 1/4 & 1/2 \\ 3/4 & 1/8 \end{bmatrix}$$

Matrices and Neural Networks



A simple neural network

$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} \xrightarrow[activation]{} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

Convolution vs Multiplication

Image processing with CNNs

A breakthrough in building models for image classification came with the discovery that a convolutional neural network (CNN) could be used to progressively extract higher- and higher-level representations of the image content. Instead of preprocessing the data to derive features like textures and shapes, a CNN takes just the image's raw pixel data as input and "learns" how to extract these features, and ultimately infer what object they constitute.

To start, the CNN receives an input feature map: a three-dimensional matrix where the size of the first two dimensions corresponds to the length and width of the images in pixels. The size of the third dimension is 3 (corresponding to the 3 channels of a color image: red, green, and blue). The CNN comprises a stack of modules, each of which performs three operations.

Convolution

A convolution extracts tiles of the input feature map, and applies filters to them to compute new features, producing an output feature map, or convolved feature (which may have a different size and depth than the input feature map). Convolutions are defined by two parameters:

1. Size of the tiles that are extracted (typically 3x3 or 5x5 pixels).
2. The depth of the output feature map, which corresponds to the number of filters that are applied.

Exercise 1 – Calculate the convolved output of the folllwing

Input Feature Map

3	5	2	8	1
9	7	5	4	3
2	0	6	1	6
6	3	7	9	2
1	4	9	5	1

Convolutional Filter

1	0	0
1	1	0
0	0	1

Rectified Linear Unit

Following each convolution operation, the CNN applies a Rectified Linear Unit (ReLU) transformation to the convolved feature, in order to introduce nonlinearity into the model. The ReLU function, F(x)=max(0,x), returns *x* for all values of $x > 0$, and returns 0 for all values of $x \leq 0$.

Pooling

After ReLU comes a pooling step, in which the CNN downsamples the convolved feature (to save on processing time), reducing the number of dimensions of the feature map, while still preserving the most critical feature information. A common algorithm used for this process is called max pooling.
Max pooling operates in a similar fashion to convolution. We slide over the feature map and extract tiles of a specified size. For each tile, the maximum value is output to a new feature map, and all other values are discarded. Max pooling operations take two parameters:

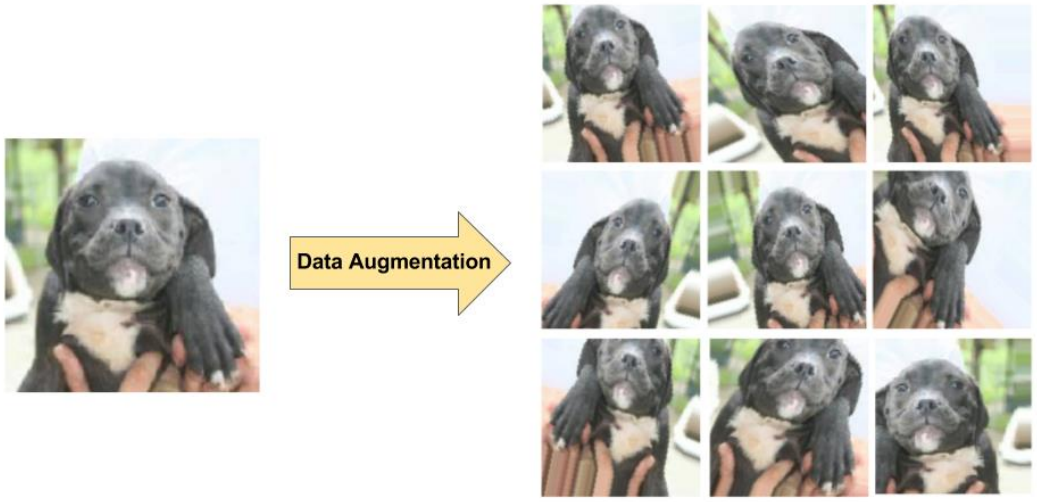
•**Size** of the max-pooling filter (typically 2x2 pixels)
•**Stride**: the distance, in pixels, separating each extracted tile. Unlike with convolution, where filters slide over the feature map pixel by pixel, in max pooling, the stride determines the locations where each tile is extracted. For a 2x2 filter, a stride of 2 specifies that the max pooling operation will extract all nonoverlapping 2x2 tiles from the feature map

Preventing overfitting

As with any machine learning model, a key concern when training a convolutional neural network is *overfitting*: a model so tuned to the specifics of the training data that it is unable to generalize to new examples. Two techniques to prevent overfitting when building a CNN are:

•**Data augmentation**: artificially boosting the diversity and number of training examples by performing random transformations to existing images to create a set of new variants (see Figure 7). Data augmentation is especially useful when the original training data set is relatively small.

•**Dropout regularization**: Randomly removing units from the neural network during a training gradient step



Pretrained Models

Training a convolutional neural network to perform image classification tasks typically requires an extremely large amount of training data, and can be very time-consuming, taking days or even weeks to complete. But what if you could leverage existing image models trained on enormous datasets, such as via TensorFlow-Slim, and adapt them for use in your own classification tasks?

One common technique for leveraging pretrained models is *feature extraction*: retrieving intermediate representations produced by the pretrained model, and then feeding these representations into a new model as input. For example, if you're training an image-classification model to distinguish different types of vegetables, you could feed training images of carrots, celery, and so on, into a pretrained model, and then extract the features from its final convolution layer, which capture all the information the model has learned about the images' higher-level attributes: color, texture, shape, etc. Then, when building your new classification model, instead of starting with raw pixels, you can use these extracted features as input, and add your fully connected classification layers on top. To increase performance when using feature extraction with a pretrained model, engineers often *fine-tune* the weight parameters applied to the extracted features.

Calculate the 2x2 maxpooled output

7	3	5	2
8	7	1	6
4	9	3	9
0	8	4	5

Resources

PyTorch script

https://colab.research.google.com/drive/1ulkdL_Fgw07tQZY-R0SWlD8fjUf8nXk8?usp=sharing#scrollTo=45z_QCDBeAyH

Notebook has been prepared by Fabien Moutarde and Guillaume Devineau from MINES ParisTech

https://people.minesparis.psl.eu/fabien.moutarde/ES_MachineLearning/Practical_deepLearning-convNets/convnet-notebook.html

CNN visualization

https://www.kaggle.com/code/thesnak/04-visualizing-what-cnn-learn-ipynb